OpenAI Codex CLI — Configuration Report

Prepared for **Murpheux** · 28 September 2026 · Verified against **Codex rust-v0.157.1** (config schema + source)
Folder: `~/agent-configs/codex/` → installs to `~/.codex/` and `~/.agents/skills/`

1. Summary

Codex 0.157 natively provides everything the design needs:

- **custom subagents** (`~/.codex/agents/*.toml`);
- **profiles as separate files**;
- skills;
- stable **hooks** using the same JSON contract as the shared guard;
- **Starlark exec-policy rules**;
- an OS sandbox;
- shell-environment **secret filtering**.

Custom prompts were **removed in 0.118**, so the 16 slash commands ship as explicit-only workflow skills (`$review`, `$plan`, …) in the Codex-only `~/.codex/skills`.

7	4	37	36	5
custom agents	tier profiles	exec-policy rules	skills	MCP servers

2. Sources used

- `openai/codex`: `codex-rs/core/config.schema.json@rust-v0.157.1`, the exec-policy parser and README, the skills loader, and the profiles/config loader. Also developers.openai.com/codex (config, rules, hooks, skills, subagents, models, Windows/WSL).
- **RoggeOhta/awesome-codex-cli**: 271 resources.

3. What was built

File	Purpose
`config.toml`	the main thread on `gpt-6-sol` (medium; plan mode high); verbosity low; `review_model = gpt-6-luna`; compaction at 180k; `tool_output_token_limit 12000`; `project_doc_max_bytes 65536`; `[agents]` defaults (Luna, 4 threads, depth 1); `on-request` + `workspace-write` with network off; `shell_environment_policy` filters; the guard hook; notifications; providers (scarlet LiteLLM with `auth.command op read`, OpenRouter); MCP
`{cheap,deep,audit,gateway}.config.toml`	Luna Q&A; Sol xhigh; Astra read-only audit (approvals never); Agentgateway routing
`AGENTS.md`	global instructions with the router policy inlined (33.5 KB; the cap is raised so it isn't truncated)
`agents/*.toml`	explorer, reviewer (Luna), planner, developer, sre (Sol; sre has network), architect (Sol xhigh), security (Astra, read-only)
`rules/guardrails.rules`	37 forbidden/prompt rules. There are deliberately no `allow` rules, because `allow` runs outside the sandbox
`hooks/guard.py` + `guardrails.json`	PreToolUse guard for Bash, shell and apply_patch; it reads file paths out of patches

`~/.agents/skills` · `~/.codex/skills`	20 portable skills · 16 workflow skills (`allow_implicit_invocation: false`)

4. Tiers (OpenAI list prices per 1M tokens, input/output)

Role	Model · effort	Price
explorer, reviewer, unnamed helpers, `/review`, `--profile cheap`	gpt-6-luna · medium/high	$0.10 / $0.50
main thread, planner, developer, sre	gpt-6-sol · medium	$2 / $10
architect, `--profile deep`	gpt-6-sol · xhigh	$2 / $10 (more reasoning tokens)
security, `--profile audit`	gpt-6-astra · medium	$10 / $50 (HIGH_RISK only)

Cached input is 10× cheaper, which is why AGENTS.md is kept as a stable prefix. `web_search = "cached"` is cheaper, and a smaller injection surface, than live search.

5. Guardrails (defence in depth)

1. **Sandbox**: `workspace-write` with network off by default (the sre agent enables network). `.git`, `.codex` and `.agents` stay read-only.
2. **Exec policy**:
   - `forbidden`: disk wipes, root/home deletes, force-push and history rewrites, destroy, bucket deletion, secret printing.
   - `prompt`: push, apply/import/state, cluster/swarm/helm mutations, opaque `bash -c` and PowerShell scripts, `ansible-playbook`.
3. **Guard hook**: catches what argv prefixes miss (`git push origin -f`, pipes into interpreters, `.env`, interactive programs, patches touching secret files).
4. **Environment filtering**: the built-in `*KEY*/*SECRET*/*TOKEN*` excludes plus 1Password, PAT, gateway, OpenRouter and AWS secret variables.
5. **Credentials**: the gateway token comes from `op read` run by Codex itself, so it never enters model context.

6. From awesome-codex-cli

Entry	Decision
VoltAgent/awesome-codex-subagents (terraform-engineer, kubernetes-specialist, sre-engineer, security-auditor…)	Adopted natively (our 7 agents); pinned models were replaced with our tiers
feiskyer/codex-settings (profile files, LiteLLM config, hardened rules)	Adopted natively
openai/skills (security-threat-model, gh-fix-ci, aspnet-core)	Install via `$skill-installer`
openai/codex-action, openai/codex-universal	Install for CI and outer container isolation
shanraisshan/codex-cli-hooks, codex-cli-best-practice	Ported to the current hook schema → guard
ccusage, tokscale	Install for cost reports
hashicorp/terraform-mcp-server, awslabs/mcp, jgraph/drawio-mcp	The first two are configured; draw.io is optional

7. Gotchas handled

- **Removed or changed in 2026**:
  - prompts removed → workflow skills;
  - a top-level `profile =` is now an error → profile files;

  - `approval_policy="untrusted"` and `wire_api="chat"` are hard errors, so neither is used.

- **Silent failures**:
  - unknown keys are ignored unless you run `codex --strict-config` (it's in the next steps);
  - untrusted repos load no AGENTS.md;
  - the default 32 KiB instructions cap would truncate AGENTS.md, so it was raised.
- **Hooks** must be trusted once in `/hooks`.
- **Rules** are prefix-only, so the hook covers reordered flags. Skill trees are CRLF-normalised, because CRLF breaks shebangs.

8. Validation

All 12 TOML files parse. The 37 rules were executed with stubs and every `match`/`not_match` example was verified. The guard handles argv arrays and `apply_patch` payloads (70 self-tests). The installer smoke test confirmed the hook command points at the installed guard, 20 + 16 skills are in place, and the hook exits 2 on a force-push.

9. Install

```
cd ~/agent-configs/codex && DRY_RUN=1 ./install.sh && ./install.sh
codex login && codex --strict-config          # then /hooks → trust the guard
codex execpolicy check --pretty --rules ~/.codex/rules/guardrails.rules -- terraform destroy
```